

Lucrarea nr. 10: **Sisteme de afișare a informației numerice**

1. Scopul lucrării

Lucrarea este destinată prezentării principalelor modalități de afișare a informației numerice, cu referințe concrete la un sistem de afișare cu 4 cifre zecimale ce funcționează pe principiul multiplexării în timp. Studiul acestui sistem de afișaj se face în două variante: o primă variantă se bazează pe implementarea cu circuite logice de complexitate medie iar cealaltă variantă presupune descrierea în limbaj VHDL și implementarea într-un CPLD. Pentru prima dată în cadrul acestui laborator se folosește conceptul de proiectare ierarhică.

2. Considerente teoretice

2.1. Codul BCD

Așa după cum este cunoscut, sistemele digitale lucrează doar cu două valori numerice: zero și unu. Din acest motiv, din punct de vedere tehnic este preferabil ca sistemul digital să lucreze într-un sistem de numerație cu baza 2 (sistemul binar). Pe de altă parte, utilizatorul (operatorul uman) este obișnuit cu sistemul zecimal. Pentru rezolvarea acestui conflict, prima idee care ne vine în minte ar fi introducerea de circuite codificatoare/decodificatoare care să facă conversia între cele două sisteme. Aceste circuite nu fac altceva decât să complice inutil partea hardware.

O soluție de compromis se pare că s-a obținut prin introducerea codului BCD (**B**inary **C**oded **D**ecimal), acesta folosește toate regulile sistemului de numerație **zecimal** dar scrierea (reprezentarea în sistemul de calcul) se face în **binar**. Cu alte cuvinte, în codul BCD, fiecare cifră zecimală este înlocuită de scrierea sa binară pe 4 biți. Codul BCD s-a dovedit atât de util încât s-au conceput circuite de numărare și circuite aritmetice capabile să lucreze în acest cod.

Pentru a pune în evidență diferențele dintre codul binar natural și codul BCD, în tabelul de mai jos sunt prezentate câteva aspecte semnificative:

	Cod BCD	Cod binar natural
Conversia spre sistemul zecimal	$\overbrace{1001}^9 \overbrace{0011}^3 = 93_{10}$	$1001\ 0011_2 = 1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 128 + 16 + 2 + 1 = 147_{10}$
Conversia din sistemul zecimal	$193_{10} = \overbrace{0001}^1 \overbrace{1001}^9 \overbrace{0011}^3_{BCD}$	$193_{10} = 11000010_2$ Rezultatul se obține prin împărțiri succesive la 2 $193:2=96$ rest 1 $96:2=48$ rest 0 $48:2=24$ rest 0 $24:2=12$ rest 0 $12:2=6$ rest 0 $6:2=3$ rest 0 $3:2=1$ rest 1 $1:2=0$ rest 1

2.2. Afișaje statice

Cea mai întâlnită metodă de afișare a unui număr zecimal constă în aprinderea sau stingerea convenabilă a unor segmente așezate după conturul cifrei 8. Cele 7 segmente pot fi realizate cu orice surse luminoase ce pot fi controlate prin mijloace electrice. Cel mai adesea se întâlnesc afișaje cu LED-uri, cu cristale lichide sau cu descărcări în gaze.

Referitor la afișajele cu LED-uri, pentru a reduce numărul de terminale dintr-o capsulă, acestea sunt fabricate în două variante: cu anod comun (AC), sau cu catod comun (KC). Schema electrică a celor două variante constructive de afișaj cu LED-uri, precum și modul de aranjare a segmentelor se prezintă în figura 1.

Comanda afișajelor cu 7 segmente se face cu ajutorul decodificatoarelor BCD - 7 segmente, circuite ce au fost studiate într-o lucrare anterioară. Trebuie precizat că există trei tipuri de decodificatoare BCD - 7 segmente:

- prima categorie afișează toate cifrele hexazecimale așa cum sunt prezentate în figura 1;

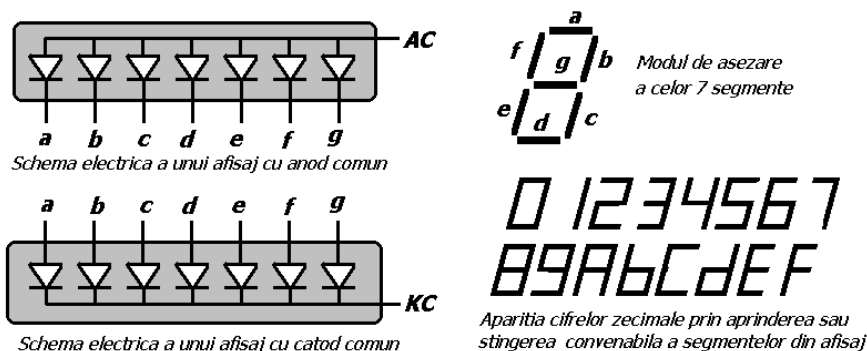


Fig.1: Afișaje cu 7 segmente implementate cu LED-uri

- b) altă categorie afișează doar cifrele zecimale de la 0 la 9, iar pentru restul codurilor mențin afișajul stins;
- c) ultima categorie afișează corect cifrele zecimale de la 0 la 9 dar, pentru restul codurilor afișează caractere mai ciudate (acest comportament se explică prin faptul că fabricantul s-a folosit de codurile non-BCD pe 4 biți pentru a reduce complexitatea circuitului).

O modalitate de a obține un afișaj cu mai multe cifre, altfel spus cu mai mulți digiți, se arată în figura 2. Se observă că fiecare cifră zecimală are propriul sau decodificator BCD-7segmente, iar informația ce trebuie afișată (spre exemplu rezultatul unei măsurători) trebuie să fie în format BCD.

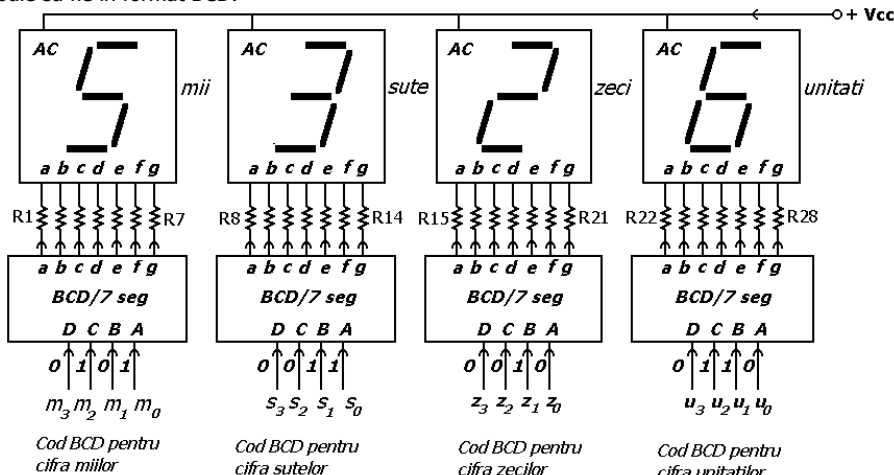


Fig.2. Schema bloc de principiu a unui afișaj static cu 4 digiți

Observații:

- decodificatoarele BCD-7segmente trebuie să aibă ieșirile active pe zero logic dacă afișajele au anodul comun;
- decodificatoarele BCD-7segmente trebuie să aibă ieșirile active pe unu logic dacă afișajele au catodul comun;
- rezistențele R1 ÷ R28 au rolul de a limita curentul prin LED-uri, fără aceste rezistențe există riscul distrugerii LED-urilor din afișaje;
- acest mod de lucru este neeconomic din punct de vedere al numărului de componente utilizat.

2.3. Afișaje dinamice

Înainte de expunerea principiului de funcționare al afișării dinamice trebuie să facem o precizare legată de comportamentul ochiului uman: **dacă frecvența de stingere/aprindere a unei surse luminoase este peste o anumită limită (denumită valoare critică), ochiul percepe respectiva sursă ca fiind aprinsă în permanență.** În urma studiilor efectuate de specialiști, s-a constatat că această valoare limită este aproximativ 46Hz. În tehnică, din motive de siguranță, se consideră o valoare de 50Hz. Această particularitate a ochiului uman este exploatată de multe sisteme tehnice, cele mai cunoscute fiind cinematografia și televiziunea.

În cazul sistemelor cu afișare dinamică, se procedează astfel:

- cifrele nu mai sunt aprinse toate odată;
- în fiecare moment de timp este aprinsă doar o singură cifră;
- fiecare cifră este menținută aprinsă un interval scurt de timp (câteva milisecunde), același pentru toate cifrele;
- cifrele se aprind pe rând: un interval de timp cifra unităților, următorul interval de timp cifra zecilor și așa mai departe până când se ajunge la ultima cifră din afișaj după care procesul se repetă;
- dacă trecerea de la o cifră la alta se face suficient de repede, ochiul percepe întreg afișajul aprins. Spre exemplu, pentru un afișaj cu 4 digiți, frecvența de trece de la o cifră la alta trebuie să fie de cel puțin 200Hz (4 cifre × 50 Hz). Aceasta înseamnă că fiecare cifră este aprinsă un interval de timp egal cu $(1/200)s = 5ms$.

Schema bloc de principiu a unui afișaj dinamic cu 4 digiți se prezintă în figura 3. Facem precizarea că acest mod de lucru este adoptat mai ales pentru sistemele de măsură realizate în structuri integrate.

Funcționare:

- Oscilatorul este folosit pentru generarea unui semnal digital cu frecvența de 200Hz. Acest semnal, denumit semnal de ceas, indică momentele de timp în care trebuie făcută trecerea de la o cifră la alta;
- Semnalul cu frecvența de 200Hz, este preluat de un numărător binar care generează următoarea secvență ciclică de coduri binare pe 2 biți: 00 → 01 → 10 → 11 → 00 → 01 ..., așa cum se prezintă și în figura 4.
- Fiecare cod binar este menținut neschimbat un interval de timp egal cu o perioadă a semnalului dat de oscilator, în cazul de față 5ms (vezi figura 4).
- Tranzistoarele pnp din figura 3 sunt folosite pe post de comutatoare electronice, ele cuplează sau decuplează anodul comun al digiților la polul pozitiv al tensiunii de alimentare.
- Tranzistoarele lucrează în regim de comutație (regim special în care tranzistorul prezintă doar două stări: tranzistor saturat = contact electric închis, respectiv tranzistor blocat = contact electric deschis).

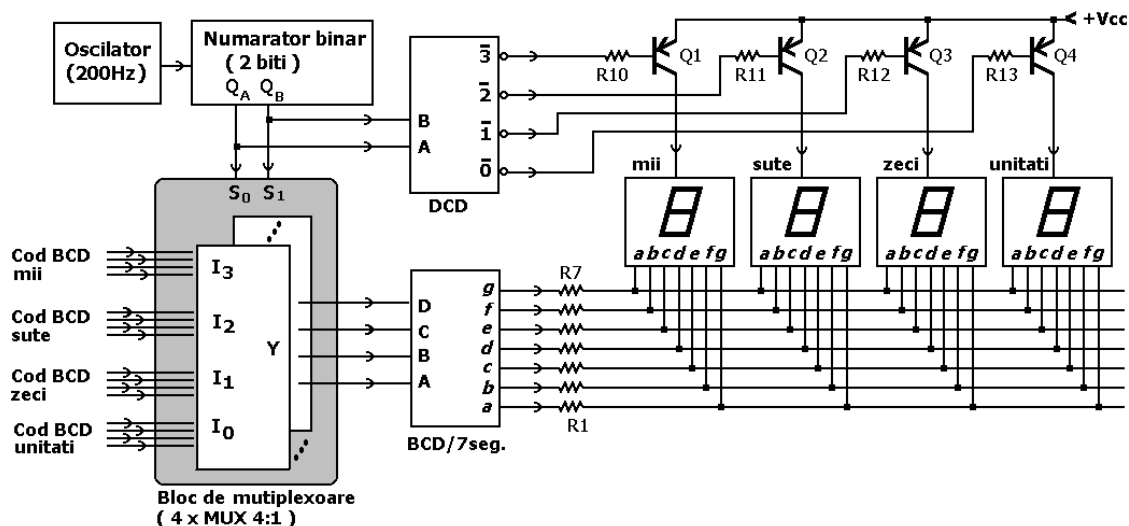


Fig. 3. Schema bloc de principiu a unui afișaj cu 4 digiți folosind multiplexarea în timp

- Pentru modul de conectare al tranzistoarelor din figura 3, blocarea se face prin aplicarea unui unu logic în bază iar saturarea prin aplicarea unui zero logic în bază.
- Deoarece comanda în bază pentru tranzistoare se face de către un DCD cu ieșirile active pe zero logic, rezultă că, în fiecare moment de timp, vom avea doar un singur tranzistor saturat iar restul vor fi blocate. Aceasta înseamnă că numai un singur digit din afișaj este conectat la polul pozitiv al tensiunii de alimentare.

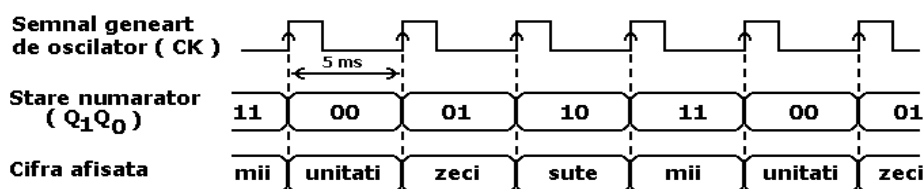


Fig. 4. Succesiunea de afișare a cifrelor zecimale pentru afișajul din figura 3.

- **Continuăm explicarea funcționării schemei din figura 3 considerând că starea număratorului este $Q_1Q_0 = 01$**
- Decodificatorul binar primește pe intrările de selecție codul BA=01, situație în care starea ieșirilor devine: $\bar{0} = 1, \bar{1} = 0, \bar{2} = 1, \bar{3} = 1$. Ținând cont de explicațiile anterioare, această stare a ieșirilor DCD, conduce la saturarea tranzistorului Q3 și la blocarea celorlalte. Așadar, la polul pozitiv al sursei de alimentare este conectat doar digitul de zeci, deci numai el se poate aprinde, supunem că digitul de zeci este activat.
- Blocul de multiplexoare primește pe intrările de date patru coduri BCD (pentru mii, sute, zeci și unități) și scoate pe ieșirile un singur cod BCD, cel care corespunde digitului activat.
- Trebuie să existe o corespondență între digitul activat și codul BCD selectat de blocul de multiplexoare: este necesar ca digitul de unități să primească codul BCD al unităților, digitul de zeci să primească codul BCD al zecilor și așa mai departe. Acest lucru se obține folosind pentru comanda intrărilor de selecție tot starea Q_1Q_0 a număratorului.
- În exemplul considerat, deoarece am presupus $Q_1Q_0 = 01$, rezultă că $S_1S_0 = 01$, deci codul BCD conectat la intrările I_1 ale multiplexoarelor vor avea cale liberă să ajungă la decodificatorul BCD/7segmente.
- După decodificare, informația despre zeci este aplicată în același timp celor patru afișaje. La prima vedere s-ar părea că informația de zeci va fi afișată pe toate cele patru afișaje. Acest lucru nu se întâmplă deoarece, așa după cum am arătat anterior, numai digitul de zeci are anodul comun conectat la +Vcc, deci numai acesta se poate aprinde.
- Digitul de zeci este menținut aprins 5ms, atâta timp cât durează o stare a număratorului. La următoarea tranziție pozitivă a semnalului de ceas, starea număratorului se schimbă și devine $Q_1Q_0 = 10$, ceea ce înseamnă că se activează digitul de sute. Așa cum se arată și în figura 4, urmează activarea miilor, a unităților, și procesul se reia.
- Modul de conectare a blocului de multiplexoare este prezentat în figura 5. Se observă că toate multiplexoarele primesc aceeași informație de selecție. Fiecare multiplexor extrage bitul cu aceeași pondere din codul de mii, de sute, de zeci respectiv de unități.
- Afișarea dinamică, denumită uneori și **afișare multiplexată**, prezintă avantajul unui consum energetic mai mic și necesită un singur decodificator BCD-7segmente.
- Rezistențele R10 ÷ R13 au rol de protecție a joncțiunii BE a tranzistoarelor, ele trebuie calculate astfel încât să permită intrarea în saturare a tranzistoarelor.
- Rezistențele R1 ÷ R7 au rol de limitare a curentului prin LED-urile afișajului.

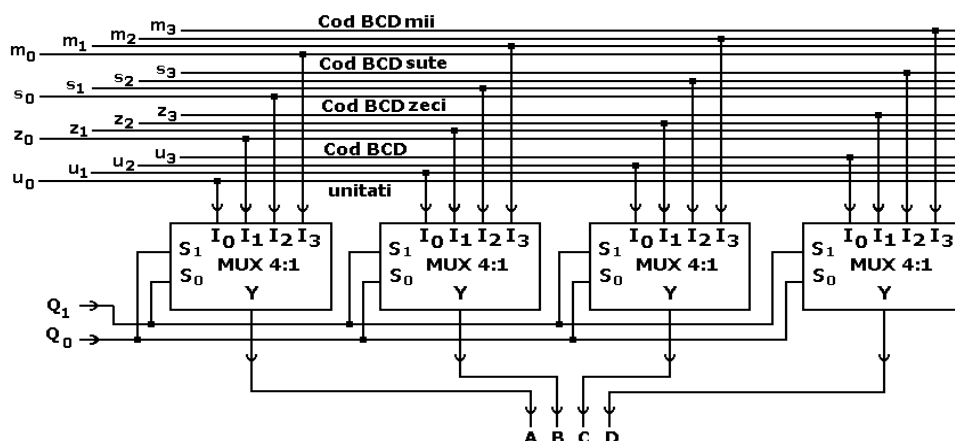


Fig. 5. Schema de conexiuni a blocului de multiplexoare

În final, dacă facem o comparație între numărul de componente necesare afișării statice (figura 3) și numărul de componente necesare afișării dinamice (figurile 4 și 5), se observă un necesar mai mare de componente pentru afișarea dinamică. Din simpla contorizare a numărului de componente se poate trage concluzia greșită că afișarea dinamică este neeconomică. În realitate, complexitatea a 4 decodificatoare BCD-7segmente este mult mai mare decât complexitatea restului de componente ce intervin în afișarea dinamică. Așadar, necesarul de arie dintr-un circuit integrat este mai mic pentru afișarea dinamică decât pentru afișarea statică, în plus mai apare și avantajul unui consum energetic mai redus atunci când discutăm de afișarea dinamică.

2.4. Descrierea unui sistem de afișare cu ajutorul VHDL – conceptul de descriere ierarhică

Din experiența proiectanților de sisteme logice complexe rezultă că proiectarea acestora se face mai ușor folosind conceptul de proiectare ierarhică. Într-o astfel de abordare, se concepe mai întâi o schema bloc a sistemului iar mai apoi se trece la proiectarea în detaliu a fiecărui bloc în parte.

În cazul utilizării limbajelor de descriere hardware, cum este și cazul VHDL, proiectarea ierarhică presupune utilizarea unui proiect în care sunt incluse mai multe fișiere :

- Un fișier de nivel înalt (*top level*), care specifică modul de interconectare a blocurilor componente din structura sistemului. Pentru această scop, se poate folosi un fișier de tip VHDL sau o descriere grafică a schemei bloc, prin intermediul editorului de scheme.
- Mai multe componente de nivel redus (*low level*). Fiecare bloc funcțional din componența schemei bloc este descris fie de un fișier VHDL, fie de o schemă logică.
- Cel puțin un fișier de constrângeri prin care sunt specificate date referitoare la implementarea

Pentru a înțelege mai bine modul de lucru vom considera ca exemplu cazul sistemului de afișare a informației numerice din figura 3. Pentru a putea fi implementată în CPLD, schema bloc trebuie puțin modificată pentru a ține cont de particularitățile hardware ale machetei de laborator. Printre aceste particularități amintim:

- oscilatorul de pe macheta de laborator are frecvența de 25,175MHz $\Rightarrow$ necesitatea introducerii unui divizor de frecvență care să primească la intrare un semnal digital cu frecvența de 25,175MHz și să furnizeze la ieșire un semnal cu frecvența de 200Hz, sau puțin mai mare;
- comanda activării afișajelor se face prin unu logic $\Rightarrow$ ieșirile decodificatorului binar DCD trebuie să fie active pe unu logic;
- comanda segmentelor se face prin zero logic $\Rightarrow$ ieșirile decodificatorului BCD/7 segmente trebuie să fie active pe zero logic ;
- macheta dispune numai de 8 switch-uri $\Rightarrow$ putem asigura informație BCD doar pentru doi digiți.

Dacă dorim ca informația afișată să fie introdusă manual avem nevoie de 16 switch-uri însă macheta nu are decât 8. Pentru a rezolva această problemă admitem ca fiecare pachet de 4 switch-uri să comande doi digiți: SW1÷SW4 vor furniza informație pentru digitul de mii și cel de sute iar SW5÷SW8 vor furniza informație pentru digitul de zeci și cel de unități.

♦ **Fișierul de nivel înalt**, să-l denumim **afisaj_4dig.vhd** este folosit pentru descrierea schemei bloc a sistemului de afișare. Din exemplul de descriere de mai jos se observă că fiecare bloc funcțional este introdus prin intermediul unei declarații de tip **component**.

Observații	Codul VHDL pentru schema bloc
Secțiune dedicată includerii de librării	<pre>library IEEE; use IEEE.std_logic_1164.all; use IEEE.std_logic_arith.all; use IEEE.std_logic_unsigned.all;</pre>
Secțiune dedicată descrierii entităților. În cazul de față:	<pre>entity afisaj_4dig is port (pin_clk : in std_logic; pin_act_dig:out std_logic_vector(3 downto 0); pin_act_seg:out std_logic_vector(6 downto 0); pin u, pin s : in std_logic_vector(3 downto 0));</pre>

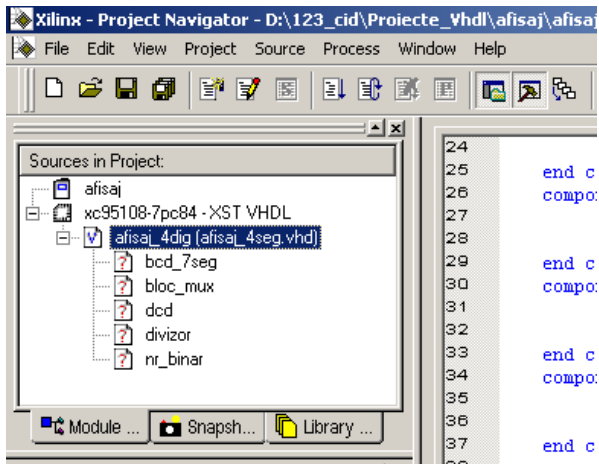
Secțiune dedicată descrierii arhitecturii. - Fiecare bloc funcțional este introdus printr-o declarație de tip component; - Pentru fiecare componentă trebuie introdus un fișier separat în care se descrie funcția logică realizată (se poate face apel la orice metodă de descriere admisă); - Denumirea intrărilor și ieșirilor trebuie să fie aceeași în declararea componentei și în descrierea funcționării acesteia; - Legăturile dintre componente (blocuri funcționale) se face prin intermediul semnalelor; - Pentru intrările sau ieșirile blocurilor funcționale (componentelor) ce sunt conectate direct la intrările sau ieșirile arhitecturii de nivel înalt, definirea de semnale nu mai este necesară; - Deși nu este recomandată, mai ales pentru începători, este permisă și declararea de componente ce conțin două sau mai multe blocuri funcționale; - Declararea componentelor și a semnalelor trebuie făcută înainte de begin –ul arhitecturii; - Pentru fiecare instanță a unei componente se declară modul de conectare a intrărilor și a ieșirilor prin port map;	<pre> end afisaj_4dig; architecture arh_afisaj_4dig of afisaj_4dig is component divizor port (f_in : in std_logic; f_out : out std_logic); end component; component nr_binar port (nr_clk : in std_logic; nr_out : out std_logic_vector(1 downto 0)); end component; component dcd port (sel_dcd : in std_logic_vector(1 downto 0); dcd_out : out std_logic_vector(3 downto 0)); end component; component bcd_7seg port (bcd_in : in std_logic_vector(3 downto 0); out_7seg_out : out std_logic_vector(6 downto 0)); end component; component bloc_mux port (u, z, s, m : in std_logic_vector(3 downto 0); sel_mux : in std_logic_vector(1 downto 0); mux_out : out std_logic_vector(3 downto 0)); end component; signal s_div, s_bcd : std_logic; signal s_num : std_logic_vector(1 downto 0); signal s_bcd : std_logic_vector(3 downto 0); begin bloc1: divizor port map (f_in => pin_clk, f_out => s_div); bloc2: nr_binar port map (nr_clk => s_div, nr_out => s_num); bloc3: dcd port map (sel_dcd => s_num, dcd_out => pin_act_dig); bloc4: bloc_mux port map (sel_mux => s_num, mux_out => s_bcd, u => pin_u, z => pin_z, s => pin_s, m => pin_m); bloc5: bcd_7seg port map (bcd_in => s_bcd, 7seg_out => pin_act_seg); end arh_afisaj_4dig; </pre>
--	---

După introducerea și salvarea acestui fișier, fereastra mediului WebPack ce indică sursele implicate în proiect arată ca în figura alăturată.

Se observă că mediul de dezvoltare a aplicației a sesizat prezența a 5 componente și acum așteaptă introducerea unei modalități acceptate pentru descrierea funcționării fiecărei componente.

Pentru a introduce codul VHDL pentru blocul de multiplexoare, este necesar să facem dublu clic pe **bloc_mux**, iar din fereastra apărută să alegem opțiunea **VHDL Module**. În continuare mediul WebPack ne cere să introducem denumirea și tipul intrărilor și ieșirilor din modul. După aceasta, se generează un template în care noi nu mai trebuie decât să descriem funcționarea respectivului circuit sau bloc logic.

♦ **Fișierul bloc_mux.vhd**, este folosit pentru descrierea funcționării blocului de multiplexoare din figura 3. O descriere posibilă este prezentată în tabelul de mai jos.



Observații	Codul VHDL pentru blocul de multiplexoare
Secțiune dedicată includerii de librării	<pre> library IEEE; use IEEE.std_logic_1164.all; use IEEE.std_logic_arith.all; use IEEE.std_logic_unsigned.all; </pre>
Secțiune dedicată descrierii entității. - Denumirea și dimensiune intrărilor și a ieșirilor trebuie să fie identică cu cea declarată în fișierul principal (afisaj_4dig);	<pre> entity bloc_mux is port (u, z, s, m : in std_logic_vector(3 downto 0); sel_mux : in std_logic_vector(1 downto 0); mux_out : out std_logic_vector(3 downto 0)); end bloc_mux; </pre>
Secțiune dedicată descrierii arhitecturii.	<pre> architecture arh_mux of bloc_mux is </pre>

- Se utilizează o descriere comportamentală a blocului de multiplexoare; - Pentru combinația de selecție 00 se alege codul unităților, pentru combinația 01 codul yecilor, etc.;	<pre> begin with sel_mux select mux_out<= u when "00", z when "01", s when "10", m when "11", u when others; end arh_mux;</pre>
---	--

♦ Fișierele **divizor.vhd**, **nr_binar.vhd** și **dcd.vhd** se introduc într-o manieră similară cu **bloc_mux.vhd**.

♦ **Fișierul de constrângeri** are extensia **.ucf** și este în principal folosit pentru a specifica pinii circuitului CPLD la care sunt conectate intrările, respectiv ieșirile afișajului.

Pentru a introduce fișierul de constrângeri se alege: **Project** → **New Source** → **Implementation Constrains File** → se alege un nume pentru fișier, spre exemplu **abc** → se face asocierea cu fișierul principal **afisaj_4dig** → se alege **Finish**. În urma acestor operații în fereastra surselor implicate în proiect apare o nouă componentă **abc.ucf**.

Dacă se face dublu clic pe **abc.ucf** se lansează un utilitar care ne ajută să edităm fișierul de constrângeri. Din fereastra apărută se selectează tabul **Ports**. După această alegere, pe ecran apare lista porturilor de intrare și de ieșire ale sistemului digital proiectat de noi. Pentru fiecare port trebuie să specificăm pinul CPLD-ului unde dorim conectarea respectivului port, aceasta presupune completarea coloanei denumită **Location**. După completare se face o salvare a fișierului de constrângeri.

Trebuie avut grijă ca numărul pinului completat în coloana Location să corespundă cu schema hardware a machetei de laborator. Pentru acesta este absolut obligatoriu să consultați tabelul de conexiuni prezentat în anexe.

Ținând cont de schema electrică a machetei de laborator, pentru afișajul pe 4 digiți descris anterior, conținutul fișierului de constrângeri este prezentat pe coloana alăturată.

```

NET "pin_act_dig<0>" LOC =
"P70";
NET "pin_act_dig<1>" LOC =
"P68";
NET "pin_act_dig<2>" LOC =
"P66";
NET "pin_act_dig<3>" LOC =
"P63";
NET "pin_act_seg<0>" LOC =
"P39";
NET "pin_act_seg<1>" LOC =
"P41";
NET "pin_act_seg<2>" LOC =
"P44";
NET "pin_act_seg<3>" LOC =
"P46";
NET "pin_act_seg<4>" LOC =
"P48";
NET "pin_act_seg<5>" LOC =
"P51";
NET "pin_act_seg<6>" LOC =
"P53";
NET "pin_s<3>" LOC = "P37";
NET "pin_s<2>" LOC = "P40";
NET "pin_s<1>" LOC = "P43";
NET "pin_s<0>" LOC = "P45";
NET "pin_u<3>" LOC = "P47";
NET "pin_u<2>" LOC = "P50";
NET "pin_u<1>" LOC = "P52";
NET "pin_u<0>" LOC = "P54";
```

♦ **În implementarea circuitului în CPLD.** După introducerea surselor în proiect (etapă denumită **Design Entry**), urmează etapa de sinteză (**Synthesis**) - în care se generează o schemă logică pe baza descrierilor din etapa anterioară, apoi o etapă de implementare (**Implement Design**) - în care circuitul logic deja sintetizat este amplasat în CPLD ținând cont de resursele acestuia și de cerințele utilizatorului. Ultima etapă constă în generarea fișierelor de configurare a circuitului CPLD, etapă denumită incorect de generare a fișierelor de programare (**Generate Programming Files**).

Cu excepția etapei de **Design Entry**, restul etapelor se fac în mod automat de către mediul WebPack (nu necesită nici o intervenție din partea utilizatorului).

Pentru generarea fișierului de configurare, în fereastra resurselor se alege fișierul principal (**afisaj_4dig**) iar în fereastra proceselor disponibile pentru acest fișier se face dublu clic pe opțiunea **Configure Device (IMPACT)**. În ferestre ce urmează se aleg următoarele opțiuni: **Configure Devices** → **Next** → **Boundary Scan Mode** → **Automatically connect to cable ...** → **Finish** → **OK** → se alege fișierul **afisaj_4dig.jed** → **Open** → clic dreapta pe icoana circuitului XC95108 → **Program...** → se alege opțiunea **Erase Before Programming** → **OK**. Dacă totul este în regulă, după câteva secunde va apare mesajul **Programming Succeeded** și se poate trece la verificarea funcționării circuitului.



3. Desfășurarea lucrării

3.1. Afișarea statică.

- A. Referitor la schema din figura 2, răspundeți la următoarele întrebări:
- Care este căderea de tensiune pe un LED aflat în conducție ?

- O reducere a necesarului de rezistențe de limitare se poate obține eliminând rezistențele de limitare a curentului dintre decodificatoare și afișaje și montarea altora între V_{cc} și anodul comun. Este posibilă o astfel de abordare? Care ar fi inconvenientele?

3.2. Afișarea dinamică.

A. Referitor la schema din figura 3, răspundeți la următoarele întrebări:

- Ce se întâmplă dacă frecvența oscilatorului este de 400Hz? Dar dacă este de 150Hz?
- Ce modificări trebuie făcute în schemă pentru a obține un afișaj cu 6 digiți?
- Dacă avem la dispoziție un decodificator BCD-7segmente cu ieșiri active pe unu logic și afișaje cu anodul comun, ce modificări trebuie efectuate în schemă pentru a deveni funcțională?
- Este strict necesară "aprinderea" cifrelor în ordinea: unități, zeci, sute, mii?
- Are importanță starea inițială de la care pleacă numărătorul?
- Care este schema pentru un afișaj cu 4 digiți cu catod comun?
- După verificarea unui montaj electronic s-a constatat că există o eroare de execuție: ieșirea $\bar{2}$ comandă tranzistorul Q3 iar ieșirea $\bar{1}$ comandă tranzistorul Q2. Cum trebuie conectate codurile BCD de la intrarea blocului de multiplexoare pentru ca informația de intrare să fie afișată corect?
- Care este schema logică realizată cu porți NAND ce poate înlocui decodificatorul binar ce comandă baza tranzistoarelor?
- Care este intervalul maxim de timp (în cazul cel mai defavorabil) dintre modificarea informației de intrare și afișarea efectivă a modificării?
- Care este efectul vizibil dacă se întrerupe legătura dintre ieșirea Q_B a numărătorului și intrarea de selecție B a decodificatorului binar?

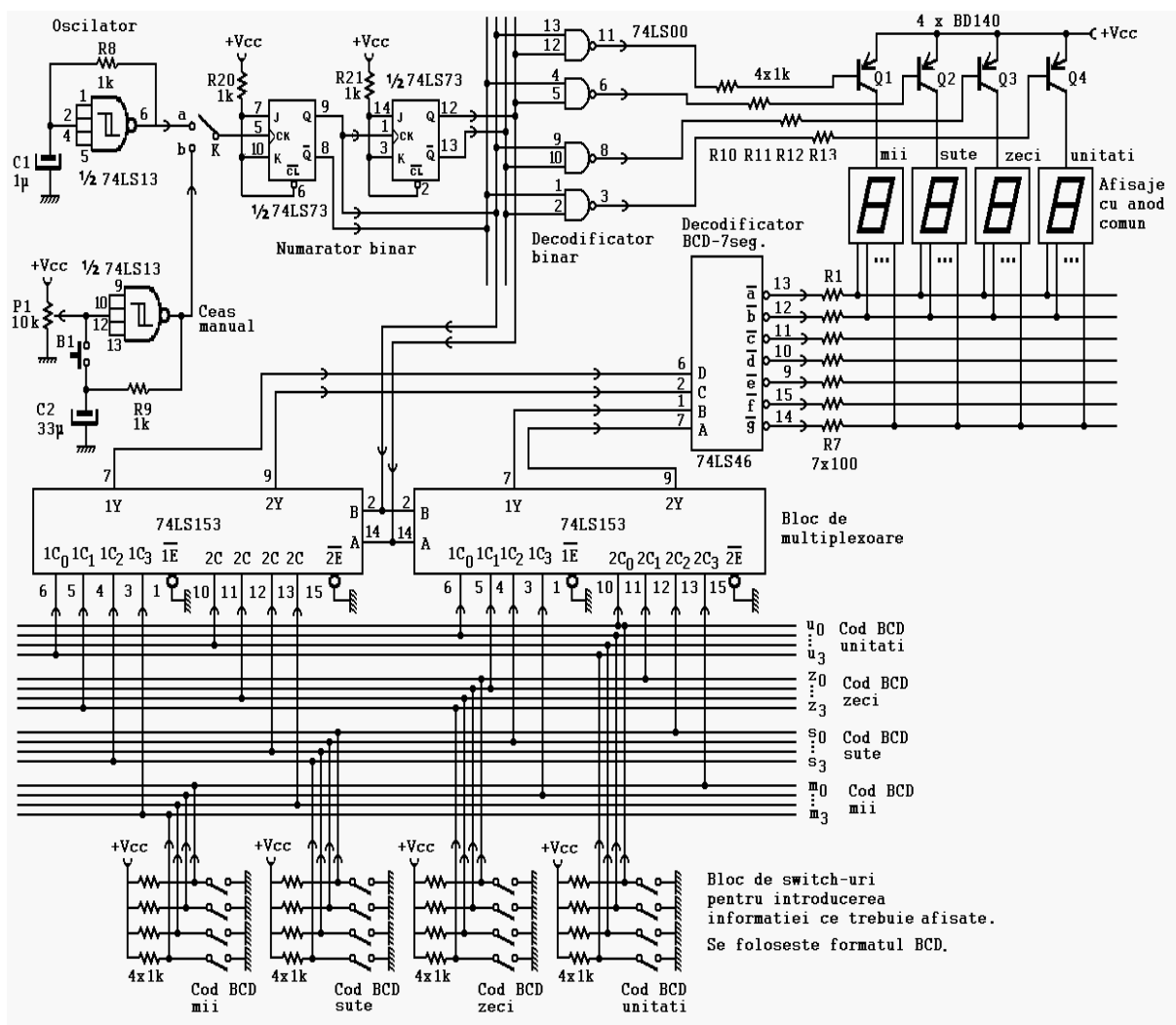


Fig.6. Schema electrică a machetei de laborator reprezentând un afișaj multiplexat cu 4 digiți

B. Referitor la schema electrică a machetei de laborator prezentată în figura 6, răspundeți la următoarele întrebări:

- Care este corespondența dintre schema bloc din figura 3 și schema detaliată din figura 6?
- Cum se poate modifica frecvența oscilatorului?

- Poarta NAND cu caracteristică de transfer de tip trigger Schmitt poate fi înlocuită cu o poartă NAND cu caracteristică normală ? Dar cu un inversor cu caracteristică trigger Schmitt?
- Explicați funcționarea ceasului manual (P1, B1, C2, R9, poarta NAND cu caracteristică de tip trigger Schmitt) ?
- Cum a fost implementat numărătorul binar pe 2 biți ? Ce rol au rezistențele R20 și R21 ?
- Se poate înlocui numărătorul din schema 6 cu un circuit specializat (spre exemplu 7493) fără a modifica restul schemei?
- Care este modul de realizare a decodificatorului binar necesar pentru comanda tranzistoarelor ?
- Se poate înlocui decodificatorul binar cu un registru de deplasare pe 4 biți ?
- Consultați foile de catalog și precizați care sunt particularitățile circuitului 74LS153 ?

C. Determinări experimentale pe macheta de laborator cu componente discrete:

- Plasați comutatorul **K** pe poziția **a** și determinați cu osciloscopul frecvența semnalului de ceas (folosit pentru trecerea de la o cifră la alta);
- Plasați comutatorul **K** pe poziția **b** și acționați asupra butonului **B1**, urmăriți secvența de aprinderea a cifrelor de pe afișaj;
- În ambele situații acționați asupra switch-urilor de intrare și urmăriți efectul pe afișaj.

3.3. Implementarea unui sistem de afișaj folosind limbajul VHDL.

A. Folosind modul de lucru descris în secțiunea 2.4. a lucrării, se cere:

- Codul VHDL pentru restul blocurilor funcționale (divizor, numărător binar, decodificator binar, decodificator BCD-7segmente);
- Implementarea proiectului pe macheta de laborator cu CPLD;
- Verificarea funcționării proiectului;

B. Modificați proiectul anterior (prin adăugare de noi blocuri funcționale) astfel încât pe afișaj să apară, unul după altul în ordine crescătoare, numerele zecimale de la 0 la 9999. Numărătoarea va fi ciclică, după numărul 9999 se reîncepe de la zero.

Indicații:

- Este nevoie de introducerea unui numărător zecimal cu patru decade;
- Intrarea de ceas a numărătorului zecimal trebuie comandată de un semnal cu frecvență mică pentru ca observatorul să aibă timpul necesar să observe (pe afișaj) succesiunea stărilor. Pentru aceasta, o sugestie ar fi să micșorăm frecvența semnalului de 200Hz, printr-un bloc suplimentar de divizare a frecvenței.
- Ieșirile numărătorului se vor conecta la intrările de date ale blocului de multiplexoare. Semnalul de

C. Modificați proiectul de la subpunctul **B** astfel încât numărătoarea să se facă invers (de la 9999 spre 0).

D. Modificați proiectul de la subpunctul **B** astfel încât numărătoarea să se facă astfel: în sens crescător de la 133 la 777, după care în sens descrescător de la 777 la 133.

